React useRef Hook Tutorial

Introduction

The `useRef` hook is a React hook that allows you to create a mutable reference that persists across component re-renders. Unlike state variables, updating a ref does not trigger a re-render. `useRef` is commonly used for accessing DOM elements directly, storing mutable values, and keeping references to values that don't need to trigger re-renders.

It can be used to store a mutable value that does not cause a re-render when updated. It can also be used to access a DOM element directly.

Basic Usage

Here's how you use the `useRef` hook:

Example: useRef vs useState

Both hooks remember a value between renders, but they behave differently when the value changes.

Example comparing both:

```
import React, { useState, useRef } from "react";

function RefVsState() {
  const [stateCount, setStateCount] = useState(0);
  const refCount = useRef(0);

  const incrementState = () => {
    setStateCount(stateCount + 1);
    console.log("State count:", stateCount + 1, "(triggers re-render)");
  };
  const incrementRef = () => {
    refCount.current += 1;
    console.log("Ref count:", refCount.current, "(no re-render)");
  };

  console.log("Component rendered");

  return (
    <div>
      <h3>State count (triggers re-render): {stateCount}</h3>
      <h3>Ref count (no re-render): {refCount.current}</h3>

      <button onClick={incrementState}>Increment State</button>
      <button onClick={incrementRef}>Increment Ref</button>

      <p>Check console to see render behavior</p>
```

```
    </div>
  );
}

export default function App() {
  return (
    <div className="App">
      <RefVsState />
    </div>
  );
}
```

What happens:

- **useRef button:** Each click increases num1.current, but the number on the button doesn't change. Updating a ref does **not** trigger a re-render.
- **useState button:** Each click calls setNum2, which triggers a re-render, so the number updates immediately.
- If you click the useRef button several times and then click the useState button, the re-render shows the ref's latest value. The ref kept its value the whole time; the screen just hadn't been redrawn.

Another example: useRef

```
import { useState, useRef, useEffect } from "react";
import { createRoot } from "react-dom/client";

export default function App() {
  const [inputValue, setInputValue] = useState("");
  const count = useRef(0);

  useEffect(() => {
    count.current = count.current + 1;
  });

  return (
    <>
      <p>Type in the input field:</p>
      <input
        type="text"
        value={inputValue}
        onChange={(e) => setInputValue(e.target.value)}
      />
      <h1>Render Count: {count.current}</h1>
    </>
  );
}
```

Explanation: **useRef()** only returns one item. When we initialize **useRef** we set the initial value: **useRef(0)**.

It's like doing this: **const count = {current: 0}**. We can access the count by using **count.current**.

Accessing DOM Elements

`useRef` is perfect for accessing DOM elements when you need to:

Example: Focusing on an Element

```
import React, { useRef } from "react";

function TextInputWithFocusButton() {
  // Create a ref to store the input element
  const inputRef = useRef(null);

  const handleClick = () => {
    // Access the DOM element and focus it
    inputRef.current.focus();
  };

  return (
    <div>
      <input
        ref={inputRef}
        type="text"
        placeholder="Click button to focus me"
      />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
}

// Main App Component
export default function App() {
  return (
    <div className="App">
      <TextInputWithFocusButton />
    </div>
  );
}
```

Explanation: - **`useRef(null)`** creates a ref object with a `current` property initialized to **`null`** - The **`ref`** attribute on the input connects the DOM element to our ref - **`inputRef.current`** gives us direct access to the DOM element - We can call DOM methods like **`.focus()`** on the element.

useRef() only returns one item. It returns an Object called **current**.

Another Example: Scrolling to an element

```jsx
import React, { useRef } from "react";

function ScrollExample() {
  const topRef = useRef(null);
  const bottomRef = useRef(null);

  const scrollToTop = () => {
    topRef.current.scrollIntoView({ behavior: "smooth" });
  };

  const scrollToBottom = () => {
    bottomRef.current.scrollIntoView({ behavior: "smooth" });
  };

  return (
    <div>
      <div ref={topRef}>
        <h2>Top of the page</h2>
        <button onClick={scrollToBottom}>Scroll to Bottom</button>
      </div>

      <div style={{ height: "1000px", backgroundColor: "#f0f0f0" }}>
        <p>Lots of content here...</p>
      </div>

      <div ref={bottomRef}>
        <h2>Bottom of the page</h2>
        <button onClick={scrollToTop}>Scroll to Top</button>
      </div>
    </div>
  );
}

export default function App() {
  return (
    <div className="App">
      <ScrollExample />
    </div>
  );
}
```

Example: Measuring element dimensions

```
import React, { useRef, useState, useEffect } from "react";

function ElementMeasurer() {
  const divRef = useRef(null);
  const [dimensions, setDimensions] = useState({ width: 0, height: 0 });

  useEffect(() => {
    if (divRef.current) {
      const { offsetWidth, offsetHeight } = divRef.current;
      setDimensions({ width: offsetWidth, height: offsetHeight });
    }
  }, []);

  return (
    <div>
      <div
        ref={divRef}
        style={{
          width: "300px",
          height: "200px",
          backgroundColor: "lightblue",
          padding: "20px",
        }}
      >
        This div is being measured!
      </div>
      <p>Width: {dimensions.width}px</p>
      <p>Height: {dimensions.height}px</p>
    </div>
  );
}

export default function App() {
  return (
    <div className="App">
      <ElementMeasurer />
    </div>
  );
}
```

Storing Mutable Values

`useRef` can store any mutable value, not just DOM elements:

Example: Storing previous values

```
import React, { useState, useRef, useEffect } from "react";

function PreviousValueTracker() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef();

  useEffect(() => {
    // Store the current count as the previous count after render
    prevCountRef.current = count;
  });

  const prevCount = prevCountRef.current;

  return (
    <div>
      <h2>Current count: {count}</h2>
      <h3>Previous count: {prevCount}</h3>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
}

export default function App() {
  return (
    <div className="App">
      <PreviousValueTracker />
    </div>
  );
}
```

Example: Storing timer references

```
import React, { useState, useRef } from "react";

function StopwatchTimer() {
  const [seconds, setSeconds] = useState(0);
  const [isRunning, setIsRunning] = useState(false);
  const intervalRef = useRef(null);

  const startTimer = () => {
    if (!isRunning) {
      setIsRunning(true);
      intervalRef.current = setInterval(() => {
        setSeconds((prevSeconds) => prevSeconds + 1);
      }, 1000);
    }
  };
  const stopTimer = () => {
    if (isRunning) {
      setIsRunning(false);
      clearInterval(intervalRef.current);
    }
  };
  const resetTimer = () => {
    setIsRunning(false);
    setSeconds(0);
    clearInterval(intervalRef.current);
  };

  return (
    <div>
      <h2>Stopwatch: {seconds}s</h2>
      <button onClick={startTimer} disabled={isRunning}>
        Start
      </button>
      <button onClick={stopTimer} disabled={!isRunning}>
        Stop
      </button>
      <button onClick={resetTimer}>Reset</button>
    </div>
  );
}

export default function App() {
  return (
    <div className="App">
      <StopwatchTimer />
    </div>
  );
}
```

Explanation: - **`intervalRef.current`** stores the interval ID - We can clear the interval using the stored reference - The ref persists across renders without causing re-renders

useRef vs useState:

Here's when to use each:

useRef	useState
Value persists across renders	Value persists across renders
Doesn't trigger re-render	**Triggers re-render**
Mutable	Immutable (should be treated as such)
Access via `.current`	Direct access
Good for DOM refs, timers, counters	Good for UI state

Forward Refs

You can pass refs to child components using `forwardRef`:

```
import React, { useRef, forwardRef } from "react";

// Child component that accepts a forwarded ref
const CustomInput = forwardRef((props, ref) => {
  return (
    <input
      ref={ref}
      {...props}
      style={{
        padding: "10px",
        border: "2px solid #007bff",
        borderRadius: "4px",
      }}
    />
  );
});

// Parent component
function ForwardRefExample() {
  const inputRef = useRef(null);

  const focusInput = () => {
    inputRef.current.focus();
  };

  return (
    <div>
      <CustomInput ref={inputRef} placeholder="Custom input component" />
      <button onClick={focusInput}>Focus Custom Input</button>
    </div>
  );
}

export default function App() {
  return (
```

```
    <div className="App">
      <ForwardRefExample />
    </div>
  );
}
```

The **{...props}** takes all props passed to **CustomInput** and applies them directly to the **<input>** element. **forwardRef** is a React function that allows a component to **pass a ref down to a child element.** It's needed because refs don't automatically pass through components like regular props do.

Exercise

Task: Create a React component called `ImageGallery` that: - Uses `useRef` to store references to image elements - Implements a “zoom to fit” functionality that scrolls to and highlights a specific image - Has thumbnail navigation buttons - Stores the currently highlighted image index without triggering unnecessary re-renders

Requirements: - Display 5 sample images (you can use placeholder images) - Each image should have a ref attached - Include “Previous” and “Next” buttons for navigation - Highlight the current image with a border - Implement smooth scrolling to the current image - Use refs to store the current index (don’t use useState for the index)

Sample Images URLs:

```
https://picsum.photos/400/300?random=1
https://picsum.photos/400/300?random=2
https://picsum.photos/400/300?random=3
https://picsum.photos/400/300?random=4
https://picsum.photos/400/300?random=5
```

Starter Code:

```
import React, { useRef } from 'react';
function ImageGallery() {
  // Your code here
  return (
    <div style={{ padding: '20px' }}>
      <h2>Image Gallery</h2>

      {/* Navigation buttons */}
      <div style={{ marginBottom: '20px' }}>
        {/* Your navigation buttons here */}
      </div>

      {/* Image container */}
      <div style={{
        display: 'flex',
```

```
        gap: '20px',
        overflowX: 'auto',
        padding: '20px',
        border: '1px solid #ccc'
      }}>
        {/* Your images here */}
      </div>

      <p>Current image: {/* Show current index */}</p>
    </div>
  );
}
export default ImageGallery;
```

Expected Behavior: - Initially highlights the first image - “Next” button moves to next image and scrolls it into view - “Previous” button moves to previous image and scrolls it into view - Navigation wraps around (after last image, goes to first) - Current image has a visible border/highlight - Smooth scrolling animation when navigating

Hints: - Use `useRef` for image references and current index - Use `scrollIntoView({ behavior: 'smooth' })` for smooth scrolling - Store image refs in an array or individual refs - Use a counter in a ref to track current index - Force re-renders only when needed for visual updates

---

Best Practices

✅ Do this:

```
// Use refs for DOM manipulation
const inputRef = useRef(null);
inputRef.current.focus();

// Use refs for storing mutable values that don't need re-renders
const countRef = useRef(0);
countRef.current += 1;

// Initialize refs with appropriate default values
const divRef = useRef(null); // For DOM elements
const timerRef = useRef(null); // For timer IDs
const valueRef = useRef(0); // For numbers
```

❌ Avoid this:

```
// Don't use refs for values that should trigger re-renders
const countRef = useRef(0);
// This won't update the UI:
countRef.current += 1;

// Don't access refs during render (except for initialization)
function BadComponent() {
```

```
  const ref = useRef(null);
  ref.current.style.color = 'red'; // Don't do this during render
  return <div ref={ref}>Content</div>;
}

// Don't forget to check if ref.current exists
inputRef.current.focus(); // Might be null!
// Better:
if (inputRef.current) {
  inputRef.current.focus();
}
```

When to use useRef:

- **DOM manipulation** (focus, scroll, measure)
- **Storing mutable values** that don't affect rendering
- **Previous values** tracking
- **Timer/interval IDs**
- **Third-party library instances**

When NOT to use useRef:

- **UI state** that should trigger re-renders
- **Props or computed values**
- **Values that should be reactive**

Summary

- `useRef` creates a mutable reference that persists across renders
- Updating refs doesn't trigger re-renders
- Perfect for DOM access, storing timers, and mutable values
- Use `forwardRef` to pass refs to child components
- Always check if `ref.current` exists before using it
- Choose `useRef` for non-reactive data, `useState` for reactive data

Takeaway: Use useState when a change should update the screen. Use useRef when you need to store a value (or access a DOM element) without causing a re-render.

For more information, see the React documentation on useRef.